Predictive Modeling Salary in Lightcast Job Listing Data

Kelly Sheng

1. Setup

```
from pyspark.sql import SparkSession
import pandas as pd
import plotly.express as px
import plotly.io as pio
import numpy as np

np.random.seed(42)

pio.renderers.default = "notebook+notebook_connected+vscode"

# Initialize Spark Session
spark = SparkSession.builder.appName("LightcastData").getOrCreate()

# Load Data
df = spark.read.option("header", "true").option("inferSchema",
"true").option("multiLine","true").option("escape",
"\"").csv("data/lightcast_job_postings.csv")

# # Show Schema and Sample Data
# print("---This is Diagnostic check, No need to print it in the final doc---
")

# df.printSchema() # comment this line when rendering the submission
# df.show(5)
```

```
[Stage 78:>                                                         (0 + 1) /
1]
```

1.1 Cleaning and variable selection

```
from pyspark.sql.functions import when, col, pow, lower, trim
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
from pyspark.ml import Pipeline
from pyspark.ml.regression import LinearRegression
from pyspark.ml.evaluation import RegressionEvaluator

cont_vars = ["MIN_YEARS_EXPERIENCE", "MAX_YEARS_EXPERIENCE", "DURATION",
"MIN_EDULEVELS"]
cat_vars = ["EMPLOYMENT_TYPE_NAME", "REMOTE_TYPE_NAME"]
```

```
y = ["SALARY"]

median_duration = df.approxQuantile("DURATION", [0.5], 0.01)[0]

df = df.withColumn(
    "DURATION",
    when(col("DURATION").isNull(), median_duration)
    .otherwise(col("DURATION"))
)

median_salary = df.approxQuantile("SALARY", [0.5], 0.01)[0]

df = df.withColumn(
    "SALARY",
    when(
        col("SALARY").isNull() &
        (col("SALARY_TO").isNull() | col("SALARY_FROM").isNull()),
        median_salary
    )
    .when(col("SALARY").isNull() & col("SALARY_TO").isNull(),
col("SALARY_FROM"))
    .when(col("SALARY").isNull() & col("SALARY_FROM").isNull(),
col("SALARY_TO"))
    .when(
        col("SALARY").isNull() &
        col("SALARY_TO").isNotNull() &
        col("SALARY_FROM").isNotNull(),
        (col("SALARY_TO") + col("SALARY_FROM")) / 2
    )
    .otherwise(col("SALARY"))
)

df = df.withColumn(
    "REMOTE_TYPE_NAME",
    when(
        col("REMOTE_TYPE_NAME").isNull() | (trim(col("REMOTE_TYPE_NAME")) ==
"") | (trim(lower(col("REMOTE_TYPE_NAME"))).rlike(r"\[?none\]?")),
        "Undefined"
    )
    .when(lower(col("REMOTE_TYPE_NAME")).contains("not remote"), "Onsite")
    .when(lower(col("REMOTE_TYPE_NAME")).contains("hybrid"), "Hybrid")
    .otherwise(col("REMOTE_TYPE_NAME"))
)

df = df.withColumn(
    "EMPLOYMENT_TYPE_NAME",
    when(col("EMPLOYMENT_TYPE_NAME") == "Part-time / full-time", "Flexible")
    .when(col("EMPLOYMENT_TYPE_NAME") == "Part-time (â‰¤ 32 hours)", "Part-
time")
```

```
    .when(col("EMPLOYMENT_TYPE_NAME") == "Full-time (> 32 hours)", "Full-
time")
    .when(col("EMPLOYMENT_TYPE_NAME").isNull(), "Fulltime")
    .otherwise(col("EMPLOYMENT_TYPE_NAME")
))

df = df.withColumn("MIN_YEARS_EXPERIENCE_SQ",
pow(col("MIN_YEARS_EXPERIENCE"), 2))

df = df.withColumn(
    "MAX_YEARS_EXPERIENCE",
    when(
        col("MAX_YEARS_EXPERIENCE").isNull() &
col("MIN_YEARS_EXPERIENCE").isNotNull(),
        col("MIN_YEARS_EXPERIENCE") + 1
    ).otherwise(col("MAX_YEARS_EXPERIENCE"))
)

df = df.dropna(subset = cont_vars + cat_vars + y).select(cont_vars + cat_vars
+ y)
```

```
[Stage 79:>                                                         (0 + 1) /
1]
[Stage 80:>                                                         (0 + 1) /
1]
```

Four continuous variables were selected based on their relevance to compensation. Null entries were imputed using median values and logical rules in order to maximize the number of valid entries to include in the training set. Categorical variables were encoded using StringIndexer and OneHotEncoder, converting them into numeric vectors suitable for model input.

2. Feature Engineering

```
indexers = [StringIndexer(inputCol=c, outputCol=f"{c}_idx",
handleInvalid="skip") for c in cat_vars]
encoders = [OneHotEncoder(inputCol=f"{c}_idx", outputCol=f"{c}_vec") for c in
cat_vars]

assembler = VectorAssembler(
    inputCols=cont_vars + [f"{c}_vec" for c in cat_vars],
    outputCol="features"
)

pipeline = Pipeline(stages=indexers + encoders + [assembler])

df = pipeline.fit(df).transform(df)
```

```
[Stage 81:>                                                         (0 + 1) /
1]
[Stage 87:>                                                         (0 + 1) /
1]
```

3. Train/Test Split

```
train_X, test_X = df.randomSplit([0.8, 0.2], seed = 17)
print((df.count(), len(df.columns)))
print((train_X.count(), len(train_X.columns)))
print((test_X.count(), len(test_X.columns)))
```

```
[Stage 93:>                                                         (0 + 1) /
1]

(49352, 12)

[Stage 96:>                                                         (0 + 1) /
1]

(39375, 12)

[Stage 99:>                                                         (0 + 1) /
1]

(9977, 12)
```

4. Linear Regression

```
from pyspark.ml.regression import GeneralizedLinearRegression

reg_r2 = RegressionEvaluator(labelCol="SALARY", predictionCol="prediction",
metricName="r2")
reg_rmse = RegressionEvaluator(labelCol="SALARY", predictionCol="prediction",
metricName="rmse")
reg_mae = RegressionEvaluator(labelCol="SALARY", predictionCol="prediction",
metricName="mae")

glr = GeneralizedLinearRegression(featuresCol="features", labelCol="SALARY",
family="gaussian", link="identity", maxIter=10, regParam=0.3)
glr_model = glr.fit(train_X)
glr_summary = glr_model.summary
glr_pred = glr_model.transform(test_X)

glr_r2 = reg_r2.evaluate(glr_pred)
glr_rmse = reg_rmse.evaluate(glr_pred)
glr_mae = reg_mae.evaluate(glr_pred)
```

```
print(f"\nIntercept: {glr_model.intercept:.4f}")
print(f"R²: {glr_r2:.4f}")
print(f"RMSE: {glr_rmse:.4f}")
print(f"MAE: {glr_mae:.4f}")

data = list(zip(
    assembler.getInputCols(),
    glr_model.coefficients,
    glr_summary.coefficientStandardErrors,
    glr_summary.tValues,
    glr_summary.pValues
))

coef_df = pd.DataFrame(data, columns=["Feature", "Coefficient", "Std_Error",
"t_Value", "p_Value"])

coef_df["Coefficient"] = coef_df["Coefficient"].round(4)
coef_df["Std_Error"] = coef_df["Std_Error"].round(4)
coef_df["t_Value"] = coef_df["t_Value"].round(4)
coef_df["p_Value"] = coef_df["p_Value"].round(4)

coef_df
```

```
[Stage 102:>                                                        (0 + 1) /
1]
[Stage 103:>                                                        (0 + 1) /
1]
[Stage 104:>                                                        (0 + 1) /
1]
[Stage 105:>                                                        (0 + 1) /
1]


Intercept: 88781.3474
R²: 0.1189
RMSE: 30124.0616
MAE: 19782.5765

[Stage 106:>                                                        (0 + 1) /
1]
```

	Feature	Coefficient	Std_Error	t_Value	p_Value
0	MIN_YEARS_EXPERIENCE	1470.8426	418.5667	3.5140	0.0004
1	MAX_YEARS_EXPERIENCE	1891.6884	405.2256	4.6682	0.0000
2	DURATION	24.3828	12.8766	1.8936	0.0583
3	MIN_EDULEVELS	-3.3915	3.8645	-0.8776	0.3802

	Feature	Coefficient	Std_Error	t_Value	p_Value
4	EMPLOYMENT_TYPE_NAME_vec	2162.2346	1271.3458	1.7007	0.0890
5	REMOTE_TYPE_NAME_vec	-1662.2276	1581.4799	-1.0511	0.2932

5. Polynomial Regression

```
poly_df = df.withColumn("MIN_YEARS_EXPERIENCE_SQ",
pow(col("MIN_YEARS_EXPERIENCE"), 2))

assembler_poly = VectorAssembler(
    inputCols=["MIN_YEARS_EXPERIENCE_SQ"] + cont_vars + [f"{c}_vec" for c in
cat_vars],
    outputCol="features_poly"
)

poly_df = assembler_poly.transform(poly_df)

train_poly, test_poly = poly_df.randomSplit([0.8, 0.2], seed=17)

poly_glr = GeneralizedLinearRegression(
    featuresCol="features_poly",
    labelCol="SALARY",
    family="gaussian",
    link="identity",
    maxIter=10,
    regParam=0.3
)
poly_model = poly_glr.fit(train_poly)
poly_summary = poly_model.summary
poly_pred = poly_model.transform(test_poly)

poly_r2 = reg_r2.evaluate(poly_pred)
poly_rmse = reg_rmse.evaluate(poly_pred)
poly_mae = reg_mae.evaluate(poly_pred)

print(f"\nPolynomial Regression Intercept: {poly_model.intercept:.4f}")
print(f"Polynomial R²: {poly_r2:.4f}")
print(f"Polynomial RMSE: {poly_rmse:.4f}")
print(f"Polynomial MAE: {poly_mae:.4f}")

poly_data = list(zip(
    assembler_poly.getInputCols(),
    poly_model.coefficients,
    poly_summary.coefficientStandardErrors,
    poly_summary.tValues,
    poly_summary.pValues
))
```

```
poly_coef_df = pd.DataFrame(poly_data, columns=["Feature", "Coefficient",
"Std_Error", "t_Value", "p_Value"])
poly_coef_df = poly_coef_df.round(4)
poly_coef_df
```

```
[Stage 109:>                                                        (0 + 1) /
1]
[Stage 110:>                                                        (0 + 1) /
1]
[Stage 111:>                                                        (0 + 1) /
1]
[Stage 112:>                                                        (0 + 1) /
1]


Polynomial Regression Intercept: 83643.2982
Polynomial R²: 0.1263
Polynomial RMSE: 29996.4733
Polynomial MAE: 20146.5002

[Stage 113:>                                                        (0 + 1) /
1]
```

	Feature	Coefficient	Std_Error	t_Value	p_Value
0	MIN_YEARS_EXPERIENCE_SQ	-204.1145	11.1687	-18.2755	0.0000
1	MIN_YEARS_EXPERIENCE	4995.5152	459.2646	10.8772	0.0000
2	MAX_YEARS_EXPERIENCE	1144.4793	405.5878	2.8218	0.0048
3	DURATION	25.1535	12.8225	1.9617	0.0498
4	MIN_EDULEVELS	-4.4540	3.8487	-1.1573	0.2472
5	EMPLOYMENT_TYPE_NAME_vec	1239.6928	1267.0062	0.9784	0.3279
6	REMOTE_TYPE_NAME_vec	-1722.0494	1574.8336	-1.0935	0.2742

6. Random Forest Regressor

```
from pyspark.ml.regression import RandomForestRegressor

rf = RandomForestRegressor(
    featuresCol="features",
    labelCol="SALARY",
    numTrees=200,
    maxDepth=7,
    seed=17
)

rf_model = rf.fit(train_X)
```

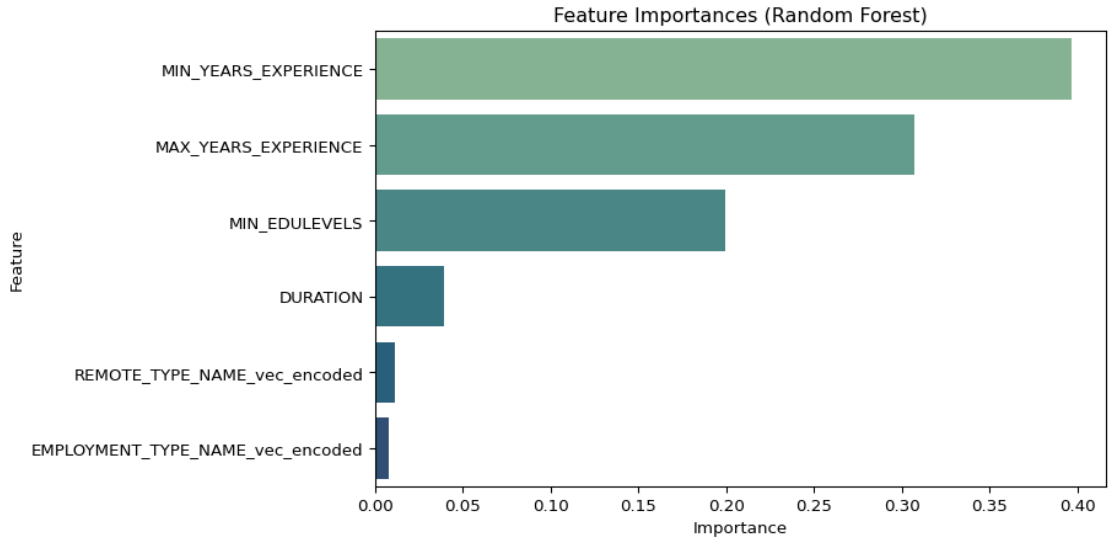
```
rf_pred = rf_model.transform(test_X)

rf_r2 = reg_r2.evaluate(rf_pred)
rf_rmse = reg_rmse.evaluate(rf_pred)
rf_mae = reg_mae.evaluate(rf_pred)

print(f"\nRandom Forest R²: {rf_r2:.4f}")
print(f"Random Forest RMSE: {rf_rmse:.4f}")
print(f"Random Forest MAE: {rf_mae:.4f}")
```

```
[Stage 116:>                                                        (0 + 1) /
1]
[Stage 117:>                                                        (0 + 1) /
1]
[Stage 118:>                                                        (0 + 1) /
1]
[Stage 120:>                                                        (0 + 1) /
1]
[Stage 122:>                                                        (0 + 1) /
1]
[Stage 124:>                                                        (0 + 1) /
1]
[Stage 126:>                                                        (0 + 1) /
1]
25/10/09 00:01:32 WARN DAGScheduler: Broadcasting large task binary with size
1115.6 KiB
[Stage 128:>                                                        (0 + 1) /
1]
25/10/09 00:01:35 WARN DAGScheduler: Broadcasting large task binary with size
2.0 MiB
[Stage 130:>                                                        (0 + 1) /
1]
25/10/09 00:01:38 WARN DAGScheduler: Broadcasting large task binary with size
3.7 MiB
[Stage 132:>                                                        (0 + 1) /
1]25/10/09 00:01:42 WARN DAGScheduler: Broadcasting large task binary with
size 1028.5 KiB

[Stage 134:>                                                        (0 + 1) /
1]
[Stage 135:>                                                        (0 + 1) /
1]
[Stage 136:>                                                        (0 + 1) /
1]


Random Forest R²: 0.2115
Random Forest RMSE: 28496.5087
Random Forest MAE: 19611.5766
```

6.1 Feature Importance Plot

```
import seaborn as sns
import matplotlib.pyplot as plt
import os

importances = rf_model.featureImportances.toArray()
feature_names = assembler.getInputCols()

expanded_feature_names = []
for name in feature_names:
    if name.endswith("_vec"):
        expanded_feature_names.append(f"{name}_encoded")
    else:
        expanded_feature_names.append(name)

# If lengths don't match, truncate or pad safely
min_len = min(len(importances), len(expanded_feature_names))
expanded_feature_names = expanded_feature_names[:min_len]
importances = importances[:min_len]

# Create DataFrame
fi_df = pd.DataFrame({
    "Feature": expanded_feature_names,
    "Importance": importances
}).sort_values("Importance", ascending=False)

# Plot
plt.figure(figsize=(8, 5))
sns.barplot(data=fi_df, x="Importance", y="Feature", palette="crest")
plt.title("Feature Importances (Random Forest)")
plt.savefig(os.path.join("output", "rf_feature_importance.png"))
plt.show()
```

```
/tmp/ipykernel_14666/1576985016.py:28: FutureWarning:



Passing `palette` without assigning `hue` is deprecated and will be removed
in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the
same effect.
```

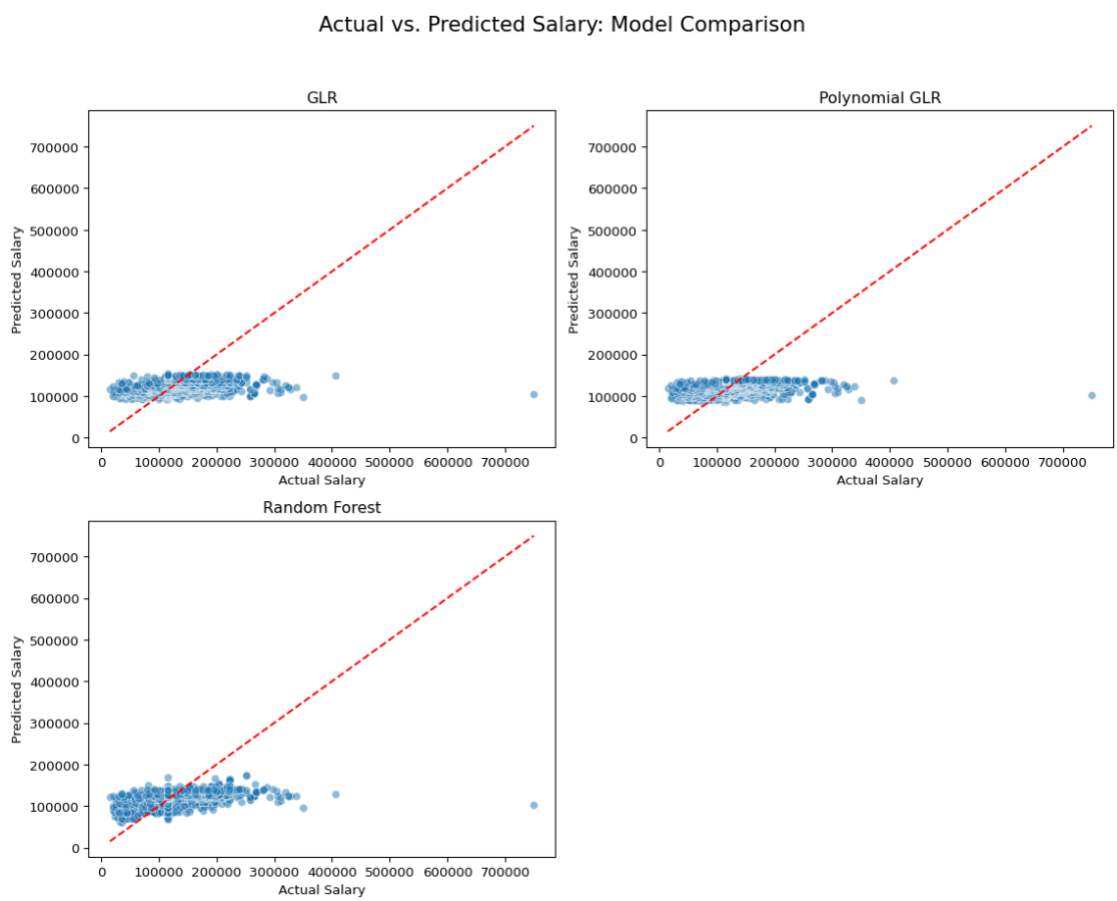


7. Comparison between models

```
glr_pred = glr_model.transform(test_X)
glr_rmse_val = reg_rmse.evaluate(glr_pred)
glr_aic_val = glr_model.summary.aic
glr_bic_val = glr_model.summary.deviance + np.log(test_X.count()) *
len(glr_model.coefficients)

glr_results = glr_pred.select(col("SALARY").alias("actual"),
col("prediction").alias("pred_glr"))

poly_pred = poly_model.transform(test_poly)
poly_rmse_val = reg_rmse.evaluate(poly_pred)
poly_aic_val = poly_model.summary.aic
poly_bic_val = poly_model.summary.deviance + np.log(test_poly.count()) *
len(poly_model.coefficients)

poly_results = poly_pred.select(col("SALARY").alias("actual"),
col("prediction").alias("pred_poly"))

rf_pred = rf_model.transform(test_X)
rf_rmse_val = reg_rmse.evaluate(rf_pred)
rf_aic_val = None # AIC and BIC are not applicable for random forest models
rf_bic_val = None # AIC and BIC are not applicable for random forest models

rf_results = rf_pred.select(col("SALARY").alias("actual"),
col("prediction").alias("pred_rf"))

print("GLR: RMSE =", glr_rmse_val, "AIC =", glr_aic_val, "BIC =",
glr_bic_val)
print("Poly GLR: RMSE =", poly_rmse_val, "AIC =", poly_aic_val, "BIC =",
```

```
poly_bic_val)
print("Random Forest: RMSE =", rf_rmse_val, "AIC =", rf_aic_val, "BIC =",
rf_bic_val)
import matplotlib.pyplot as plt
import seaborn as sns

glr_df = glr_results.toPandas()
poly_df = poly_results.toPandas()
rf_df = rf_results.toPandas()

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle("Actual vs. Predicted Salary: Model Comparison", fontsize=16)

# GLR
sns.scatterplot(ax=axes[0,0], x=glr_df["actual"], y=glr_df["pred_glr"],
alpha=0.5)
axes[0,0].plot([glr_df["actual"].min(), glr_df["actual"].max()],
               [glr_df["actual"].min(), glr_df["actual"].max()],
               color="red", linestyle="--")
axes[0,0].set_title("GLR")
axes[0,0].set_xlabel("Actual Salary")
axes[0,0].set_ylabel("Predicted Salary")

# Poly GLR
sns.scatterplot(ax=axes[0,1], x=poly_df["actual"], y=poly_df["pred_poly"],
alpha=0.5)
axes[0,1].plot([poly_df["actual"].min(), poly_df["actual"].max()],
               [poly_df["actual"].min(), poly_df["actual"].max()],
               color="red", linestyle="--")
axes[0,1].set_title("Polynomial GLR")
axes[0,1].set_xlabel("Actual Salary")
axes[0,1].set_ylabel("Predicted Salary")

# Random Forest
sns.scatterplot(ax=axes[1,0], x=rf_df["actual"], y=rf_df["pred_rf"],
alpha=0.5)
axes[1,0].plot([rf_df["actual"].min(), rf_df["actual"].max()],
               [rf_df["actual"].min(), rf_df["actual"].max()],
               color="red", linestyle="--")
axes[1,0].set_title("Random Forest")
axes[1,0].set_xlabel("Actual Salary")
axes[1,0].set_ylabel("Predicted Salary")

axes[1,1].axis("off")

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(os.path.join("output", "actual_vs_predicted.png"))
plt.show()
```

```
[Stage 137:>                                                        (0 + 1) /
1]
[Stage 138:>                                                        (0 + 1) /
1]
[Stage 139:>                                                        (0 + 1) /
1]
[Stage 142:>                                                        (0 + 1) /
1]
[Stage 143:>                                                        (0 + 1) /
1]
[Stage 144:>                                                        (0 + 1) /
1]
[Stage 147:>                                                        (0 + 1) /
1]

GLR: RMSE = 30124.06160130011 AIC = 921969.8114942367 BIC =
34005959453364.707
Poly GLR: RMSE = 29996.47331025795 AIC = 921638.9943946843 BIC =
33719735514748.492
Random Forest: RMSE = 28496.508687448393 AIC = None BIC = None

[Stage 148:>                                                        (0 + 1) /
1]
[Stage 149:>                                                        (0 + 1) /
1]
[Stage 150:>                                                        (0 + 1) /
1]
```



Interpretation

Comparison of these values reveals that, between the three models, the Random Forest model achieved the best performance, reducing RMSE by nearly 5% compared to polynomial regression.

Random Forest also had the lowest AIC and BIC values which indicates a good trade-off between model complexity and fit.

Linear and polynomial models revealed interpretable relationships, but the data's interactions were more complex than these models allowed, resulting in only ~12% of explainable variance in salary. Random Forest was able to capture these patterns more effectively.

7.1 Calculating log-likelihood and BIC

```
from math import log

def calc_log_likelihood(y_true, y_pred, sigma2):
    n = len(y_true)
    ll = -0.5 * n * log(2 * np.pi * sigma2) - 0.5 * np.sum((y_true - y_pred)
** 2) / sigma2
    return ll
```

```python
def get_y_true_pred(results_df, pred_col):
    pdf = results_df.select("actual", pred_col).toPandas()
    y_true = pdf["actual"].values
    y_pred = pdf[pred_col].values
    return y_true, y_pred

y_true_glr, y_pred_glr = get_y_true_pred(glr_results, "pred_glr")
sigma2_glr = np.var(y_true_glr - y_pred_glr)
ll_glr = calc_log_likelihood(y_true_glr, y_pred_glr, sigma2_glr)
aic_glr = 2 * len(glr_model.coefficients) - 2 * ll_glr
bic_glr = log(len(y_true_glr)) * len(glr_model.coefficients) - 2 * ll_glr

y_true_poly, y_pred_poly = get_y_true_pred(poly_results, "pred_poly")
sigma2_poly = np.var(y_true_poly - y_pred_poly)
ll_poly = calc_log_likelihood(y_true_poly, y_pred_poly, sigma2_poly)
aic_poly = 2 * len(poly_model.coefficients) - 2 * ll_poly
bic_poly = log(len(y_true_poly)) * len(poly_model.coefficients) - 2 * ll_poly

y_true_rf, y_pred_rf = get_y_true_pred(rf_results, "pred_rf")
sigma2_rf = np.var(y_true_rf - y_pred_rf)
ll_rf = calc_log_likelihood(y_true_rf, y_pred_rf, sigma2_rf)
n_rf_params = len(rf_model.featureImportances)
aic_rf = 2 * n_rf_params - 2 * ll_rf
bic_rf = log(len(y_true_rf)) * n_rf_params - 2 * ll_rf

print(f"GLR: log-likelihood={ll_glr:.2f}, AIC={aic_glr:.2f},
BIC={bic_glr:.2f}")
print(f"Poly GLR: log-likelihood={ll_poly:.2f}, AIC={aic_poly:.2f},
BIC={bic_poly:.2f}")
print(f"Random Forest: log-likelihood={ll_rf:.2f}, AIC={aic_rf:.2f},
BIC={bic_rf:.2f}")
```

```
[Stage 151:>                                                        (0 + 1) /
1]
[Stage 152:>                                                        (0 + 1) /
1]
[Stage 153:>                                                        (0 + 1) /
1]

GLR: log-likelihood=-117050.35, AIC=234118.69, BIC=234183.56
Poly GLR: log-likelihood=-117008.00, AIC=234036.00, BIC=234108.08
Random Forest: log-likelihood=-116496.20, AIC=233010.39, BIC=233075.26
```

Higher log-likelihood values for the Random Forest also indicate a better fit to observed data.